Tema 2.5

1. Refactorización y control de complejidad ciclomática

Enunciado: la función `calcularMatricula(datos)` ha sido rechazada por el analizador automático del proceso de integración continua con estos tres avisos.

```
[warning/error] complexity: Function 'calcularMatricula' has a cyclomatic
         complexity of 8 (max allowed: 5).
[warning] max-depth: Blocks are nested 4 levels deep (max allowed: 2).
[warning] no-else-return: Unnecessary 'else' after 'return'.
```

Hay que reescribir la función cumpliendo cinco cosas: comprobar los errores al principio y salir en cuanto se detecten, no pasar de un nivel de indentación, no poner `else` después de un `return`, dejar el cálculo final sin nada que lo envuelva, y que siga dando exactamente los mismos resultados.

Código original (antipatrón)

```
// ANTIPATRÓN: Alta complejidad ciclomática y ramas 'else' innecesarias
function calcularMatricula(datos) {
    let resultado;

    if (datos !== null && typeof datos === "object") {
        if (typeof datos.precioBase === "number" && datos.precioBase > 0) {
            if (["general", "desempleado", "socio"].includes(datos.
         tipoEstudiante)) {
                let descuentoPorcentaje = 0;

                if (datos.tipoEstudiante === "desempleado") {
                    descuentoPorcentaje = 0.30;
                } else {
                    if (datos.tipoEstudiante === "socio") {
                        descuentoPorcentaje = 0.15;
                    }
                }

                let total = datos.precioBase * (1 - descuentoPorcentaje);

                if (datos.codigoDescuento === "WEB2026") {
                    total = total - 10;
                }

                if (total < 0) {
                    total = 0;
                }

                resultado = { valido: true, total: total };
            } else {
```

```
                resultado = { valido: false, error:
            "Tipo de estudiante no reconocido." };
                }
            } else {
                resultado = { valido: false, error:
            "El precio base debe ser un número positivo." };
            }
        } else {
            resultado = { valido: false, error:
            "Parámetros de matrícula inválidos." };
        }

        return resultado;
}
```

Qué pasa con este código. Para llegar a lo que de verdad importa, que es calcular el importe, hay que abrir cuatro comprobaciones, una dentro de otra. Está todo tan metido hacia dentro que cuesta seguir el hilo de lo que ocurre cuando los datos son correctos. Además, el descuento se decide con un `if` dentro de un `else` cuando se podía haber puesto un `else if` y quedaba más plano. Y en lugar de devolver la respuesta en cuanto se sabe, se va guardando en la variable `resultado` para devolverla al final del todo.

Código refactorizado

```
function calcularMatricula(datos) {
    // Comprobaciones al principio: si algo no vale, se sale aquí mismo
    if (!datos || typeof datos !== "object") {
        return { valido: false, error: "Parámetros de matrícula inválidos."
        };
    }

    if (typeof datos.precioBase !== "number" || datos.precioBase <= 0) {
        return { valido: false, error:
        "El precio base debe ser un número positivo." };
    }

    if (!["general", "desempleado", "socio"].includes(datos.tipoEstudiante))
        {
        return { valido: false, error: "Tipo de estudiante no reconocido." }
        ;
    }

    // Cálculo final, ya sin nada que lo envuelva
    let descuentoPorcentaje = 0;

    if (datos.tipoEstudiante === "desempleado") {
        descuentoPorcentaje = 0.30;
    } else if (datos.tipoEstudiante === "socio") {
        descuentoPorcentaje = 0.15;
    }

    let total = datos.precioBase * (1 - descuentoPorcentaje);

    if (datos.codigoDescuento === "WEB2026") {
        total = total - 10;
    }

    if (total < 0) {
        total = 0;
```

```
    }

    return { valido: true, total: total };
}
```

Las tres primeras comprobaciones van al principio y devuelven el error en cuanto algo no cuadra, sin seguir calculando. Es la idea del ejemplo `if (!datos)` del apunte: si los datos no son válidos, la función termina ahí. Si llega hasta abajo, ya se sabe que los datos están bien y no hace falta volver a mirarlos.

Con esto tampoco queda ningún `if` dentro de otro `if`: las tres comprobaciones están al mismo nivel y el descuento se resuelve con un `else if`, que es una cadena de alternativas y no algo anidado. La indentación baja de cuatro niveles a uno.

Tampoco hay `else` de más. Como cada comprobación termina en su propio `return`, no hace falta un `else` para separarla del resto. El único `else` que queda es el del descuento, que forma parte de la propia cadena de comparaciones.

Y el cálculo se queda al final, suelto, sin ninguna condición alrededor, para que lo último que se lee sea siempre la respuesta correcta.

Sobre el resultado, no cambia nada: los tres mensajes de error son los mismos y en el mismo orden, el descuento sigue aplicando 0 al tipo `general`, 30 % al `desempleado` y 15 % al `socio`, el cupón `WEB2026` sigue restando 10 € y el importe nunca baja de cero.

Banco de pruebas

```
// Batería de pruebas: Comprobación de equivalencia funcional
console.assert(
    calcularMatricula(null).valido === false &&
    calcularMatricula(null).error === "Parámetros de matrícula inválidos.",
    "Fallo en Test 1: Manejo de entrada null"
);

console.assert(
    calcularMatricula({ precioBase: -20, tipoEstudiante: "general" }).valido
          === false &&
    calcularMatricula({ precioBase: -20, tipoEstudiante: "general" }).error
         === "El precio base debe ser un número positivo.",
    "Fallo en Test 2: Precio base negativo o cero"
);

console.assert(
    calcularMatricula({ precioBase: 100, tipoEstudiante: "extranjero" }).
         valido === false &&
    calcularMatricula({ precioBase: 100, tipoEstudiante: "extranjero" }).
         error === "Tipo de estudiante no reconocido.",
    "Fallo en Test 3: Tipo de estudiante desconocido"
);

console.assert(
    calcularMatricula({ precioBase: 100, tipoEstudiante: "desempleado",
         codigoDescuento: "WEB2026" }).total === 60,
    "Fallo en Test 4: Descuento desempleado (30%) + cupón (-10 €)"
);

console.assert(
    calcularMatricula({ precioBase: 100, tipoEstudiante: "socio" }).total
```

```
        === 85,
    "Fallo en Test 5: Descuento socio (15%) sin cupón"
);

console.assert(
    calcularMatricula({ precioBase: 5, tipoEstudiante: "general",
         codigoDescuento: "WEB2026" }).total === 0,
    "Fallo en Test 6: Suelo mínimo a cero cuando el cupón excede el precio"
);

console.log("¡Todas las pruebas unitarias se ejecutaron con éxito!");
```

Qué da cada una con la función nueva:

Con `null` entra en la primera comprobación y devuelve el error de parámetros inválidos.

Con `precioBase: -20`, la segunda comprobación ve que es un número pero no es positivo y devuelve el error del precio.

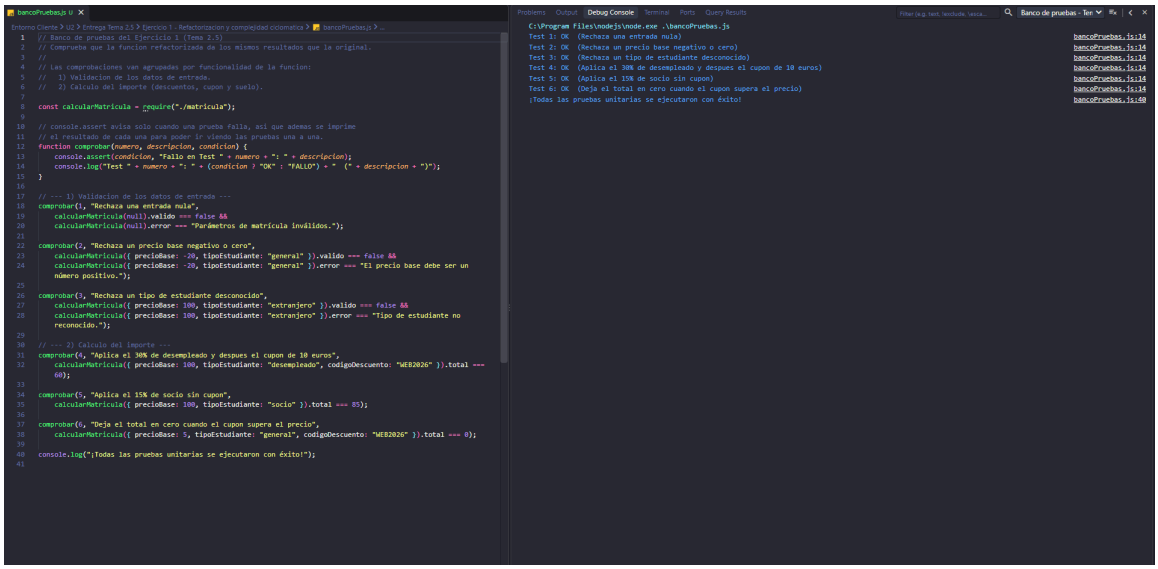
Con `tipoEstudiante: "extranjero"`, la tercera comprobación ve que no es uno de los tres tipos admitidos y devuelve el error del tipo.

Con `precioBase: 100`, `desempleado` y cupón `WEB2026`, aplica el 30 % (`100 * 0.70 = 70`) y le resta los 10 € del cupón, quedando 60.

Con `precioBase: 100` y `socio`, aplica el 15 % sin cupón (`100 * 0.85 = 85`).

Y con `precioBase: 5`, `general` y cupón `WEB2026`, sin descuento por tipo sería `5 - 10 = -5`, y el suelo lo deja en 0.

Si las seis salen bien, es que la función calcula lo mismo que antes



Ejecución del banco de pruebas en la Debug Console de Visual Studio Code: las seis comprobaciones pasan.